



Why Basic Next?

Reducing cognitive load without reducing expressive power.

Programming asks people to hold too much in their head

Modern software often makes simple ideas harder than they need to be. A developer can face terse syntax, invisible conversions, framework conventions, configuration layers, and platform-specific APIs before expressing the actual problem.

Basic Next starts from a different question: how can a language preserve expressive power while asking its reader to remember less?

The BN response

BN favors words that reveal intent, types that state a contract, and an object-oriented structure that gives behavior a home. This does not mean hiding important details. It means making important details legible.

```
CLASS Customer
  PRIVATE id AS INTEGER

  PUBLIC CONSTRUCTOR(id AS INTEGER)
    SELF.id = id
  END CONSTRUCTOR
END CLASS
```

The class boundary, visibility, constructor, and type are visible without a large amount of surrounding ceremony.

Principles in practice

- **KISS:** complexity must solve a concrete problem, not prepare for an imaginary future.
- **Clean Code and Clean Architecture:** natural outcomes of clear names, explicit dependencies, and cohesive objects - never a mandated framework.
- **Capability orientation:** cross-platform programs ask the host for a capability instead of tying their source directly to an operating system or

vendor.

- **Deliberate evolution:** the specification comes before implementation; a proposed feature must earn its place with examples and a clear semantic rule.

What BN does not promise

BN does not claim to replace established ecosystems, win performance benchmarks, or already be ready for production. Its immediate contribution is a carefully designed language foundation. Its long-term value depends on an implementation, a stable standard library, tools, and a community that keeps the core coherent.

The outcome we seek

Software should let people focus on the model they are building. BN aims for source that is easy to read months later, easy to explain in a review, and clear enough to move from idea to working code without unnecessary friction.